

If you're getting:

```
{  
  "detail": "Method Not Allowed"  
}
```

it usually means **the URL exists, but you're using the wrong HTTP method.**

For example, if your FastAPI code is:

```
@app.post("/upload")  
  
async def upload_file(file: UploadFile = File(...)):  
    return {"filename": file.filename}
```

then you **must** call it with POST:

```
curl -X POST "http://127.0.0.1:8000/upload" -F "file=@C:\temp\test.pdf"
```

Common mistake

If you do:

```
curl "http://127.0.0.1:8000/upload"
```

curl defaults to **GET**.

But your endpoint only accepts **POST**:

GET /upload ❌

POST /upload ✅

Hence:

405 Method Not Allowed

Check these 3 things

1. Your decorator

```
@app.post("/upload")
```

2. Your curl command

```
curl -X POST "http://127.0.0.1:8000/upload" -F "file=@C:\temp\test.pdf"
```

3. Your parameter name

If your function is:

```
async def upload_file(myfile: UploadFile = File(...)):
```

then curl must use:

```
-F "myfile=@C:\temp\test.pdf"
```

not:

```
-F "file=@C:\temp\test.pdf"
```

Also, if you are testing /docs, don't use curl against /docs to test the upload. /docs is the Swagger UI page. Your actual upload URL is whatever you specified in `@app.post(...)`.

If you paste your **upload function + exact curl command**, I can tell you exactly where the 405 is coming from.

```
C:\Users\shora>curl -X POST "http://127.0.0.1:8000/upload" -F  
"file=@D:\TechLearning\PythonByClaude\uploads\test.txt" {"detail":"Method Not Allowed"}
```

Your curl command is **correct**. The problem is almost certainly **your FastAPI route**, not curl.

A 405 Method Not Allowed means:

Uvicorn reached `http://127.0.0.1:8000/upload`, but the application doesn't have a **POST** route matching `/upload`.

Check your FastAPI code

You should have something like:

```
from fastapi import FastAPI, UploadFile, File
```

```
app = FastAPI()
```

```
@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    return {
        "filename": file.filename,
        "content_type": file.content_type
    }
```

Then restart Uvicorn:

```
uvicorn main:app --reload
```

And run your command again:

```
curl -X POST "http://127.0.0.1:8000/upload" -F
"file=@D:\TechLearning\PythonByClaude\uploads\test.txt"
```

The fastest way to diagnose it

Open:

```
http://127.0.0.1:8000/docs
```

Look for an endpoint:

```
POST /upload
```

You should see something like:

```
/upload
```

```
POST
```

If you **don't see** POST /upload, then your running FastAPI application doesn't contain that route.

If you see something like:

```
/upload-file
```

```
POST
```

then your curl command needs to use that exact path:

```
curl -X POST "http://127.0.0.1:8000/upload-file" -F
"file=@D:\TechLearning\PythonByClaude\uploads\test.txt"
```

One thing I want you to check

Since you've been working through FastAPI's file-upload lesson, **paste your current FastAPI code here**, especially the part containing:

```
@app.post(...)
```

I'll pinpoint why you're getting the 405 rather than making you guess.

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()
app.mount("/", StaticFiles(directory="web/static", html=True), name="static")

@app.post("/upload")
async def upload_file(file: UploadFile):
    if file is None:
        return {"status": "failed", "message": "Please upload a file", "chunks": 0}
    elif file.content_type.lower() != 'pdf':
        return {"status": "failed", "message": "Invalid file type!", "chunks": 0}
    file_path_name = f"/uploads/{file.filename}"
    with open(file_path_name, "wb") as f:
        content = await file.read()
        f.write(content)
```

I can see the exact reason for your 405 Method Not Allowed.

It is **this line**:

```
app.mount("/", StaticFiles(directory="web/static", html=True), name="static")
```

You're mounting your static files at the root / **before** defining /upload.

What is happening

Your routes effectively look like this:

```
/ → StaticFiles
```

```
/upload → POST upload_file
```

But because / is mounted first, a request to:

```
POST /upload
```

matches the static-files application first.

StaticFiles doesn't support your POST /upload operation, so you get:

```
{"detail": "Method Not Allowed"}
```

Fix

Move the static mount **after** your API routes:

```
from fastapi import FastAPI, UploadFile
```

```
from fastapi.staticfiles import StaticFiles
```

```
import utils
```

```
app = FastAPI()
```

```
@app.post("/upload")
```

```
async def upload_file(file: UploadFile):
```

```
    if file is None:
```

```
        return {
```

```
            "status": "failed",
```

```
            "message": "Please upload a file",
```

```
            "chunks": 0
```

```
        }
```

```
    elif file.content_type.lower() != "application/pdf":
```

```
        return {
```

```
            "status": "failed",
```

```
            "message": "Invalid file type!",
```

```
            "chunks": 0
```

```
        }
```

```
file_path_name = f"uploads/{file.filename}"
```

```
with open(file_path_name, "wb") as f:
```

```
    content = await file.read()
```

```
    f.write(content)
```

```
return {
```

```
    "status": "success",
```

```
    "filename": file.filename
```

```
}
```

```
app.mount("/", StaticFiles(directory="web/static", html=True), name="static")
```

Now:

POST /upload

↓

upload_file()

and everything else can fall through to:

/

↓

StaticFiles

There is another bug in your code

You currently have:

```
elif file.content_type.lower() != 'pdf':
```

That's incorrect.

For a PDF uploaded through curl or a browser, the content type is normally:

application/pdf

So use:

```
elif file.content_type.lower() != "application/pdf":
```

Otherwise a legitimate PDF will be rejected.

And one more issue: your upload path

You have:

```
file_path_name = f"/uploads/{file.filename}"
```

That leading / makes it an **absolute path** on Linux:

```
/uploads/test.pdf
```

On Windows, that's also not what you want.

If your project contains:

PythonByClaude/

|

├─ main.py

├─ uploads/

| └─

└─ web/

 └─ static/

use:

```
file_path_name = f"uploads/{file.filename}"
```

or, better:

```
from pathlib import Path
```

```
upload_dir = Path("uploads")
```

```
upload_dir.mkdir(exist_ok=True)
```

```
file_path = upload_dir / file.filename
```

```
with open(file_path, "wb") as f:
```

```
    content = await file.read()
```

```
    f.write(content)
```

Then your command should work:

```
curl -X POST "http://127.0.0.1:8000/upload" -F  
"file=@D:\TechLearning\PythonByClaude\uploads\test.txt"
```

But note: you're checking for application/pdf, while you're uploading test.txt. So after fixing the routing, you'll get Invalid file type! for that .txt file—which would be correct according to your current validation.

For testing the successful path, use an actual PDF:

```
curl -X POST "http://127.0.0.1:8000/upload" -F  
"file=@D:\TechLearning\PythonByClaude\uploads\test.pdf"
```

The key lesson here is: `app.mount("/")` is **essentially a catch-all route. Putting it before your API routes can swallow those routes.**